

embed-api

A production-minded embedding API for **multilingual-e5-large**

FastAPI · ONNX Runtime (int8) · Docker · GitHub Actions · k3s

The model shapes the API

e5 fact	What the API does about it
Trained with <code>query:</code> / <code>passage:</code> prefixes; quality drops without them	<code>input_type</code> is required ; the server adds the prefix
Reads at most 512 tokens , prefix and special tokens included	Truncates and reports <code>tokens</code> + <code>truncated</code> per input
~560M params, CPU-bound	Inference off the event loop, one request at a time, token budget per request
Output is L2-normalised, 1024-dim	Documented: cosine similarity = dot product
Hub repo is 9.5 GB, <code>main</code> moves	Pinned revision, fetch only what's needed, revision reported by <code>/v1/info</code>

What the brief asked for

- **Validation:** one pydantic schema → validation + Swagger. Size limit *before* parsing, token budget *after* tokenising.
- **Errors:** one envelope with a stable `code` and the `request_id`, validation included.
- **Logging:** JSON, one access line per request, `X-Request-ID` everywhere. **Input text is never logged** (tested).
- **Practice:** `Embedder` protocol → fast tests with a fake model; slow tests on the real one; ruff + CI.

Request lifecycle

```
client → RequestContext (request id, access log)
      → BodySizeLimit (413 before parsing)
      → pydantic validation (422)
      → readiness gate (503 while the model loads)
      → tokenise in a worker thread → token budget (422)
      → semaphore → encode in a worker thread → response
```

- Model loads **in the background**: liveness passes at once, readiness flips when warm
- Inference threads follow the **container CPU limit**, not the host's cores

Faster inference: measure speed *and* quality

Backend	Weights	1 text	32 texts	Cosine vs fp32 (min)
PyTorch fp32	2.2 GB	115 ms	1.83 s	1.000
ONNX Runtime fp32	2.2 GB	58 ms	1.82 s	1.000
ONNX Runtime int8	0.56 GB	33 ms	1.04 s	0.993

- Runtime halves single-text latency; **int8** takes another ~40% and gives **1.8× throughput**
- Quantised for **AVX2** (the deployment CPU); the Hub's int8 file needs AVX-512 VNNI
- Runtime is ONNX Runtime + tokenizers: **no torch**, image 1.8 → 0.6 GB

A bug worth finding

Dynamic int8 quantisation scales activations **per input tensor**.

Batched, a text's embedding depended on its neighbours in the request: cosine **0.994** to itself embedded alone.

→ Same text, different vector: breaks caching and deduplication.

Fix: one text per forward pass (−10% throughput, no wasted padding) + a test that asserts batch invariance.

Shipping it

- **Docker:** int8 model built in a separate stage from a pinned revision; runtime has no torch; non-root, read-only root fs
- **CI:** ruff + tests → build → **smoke-test the container** → push to GHCR
- **CD:** GitOps on my k3s homelab. Image updater + ArgoCD roll out each new digest. On the node: ready in 1.9 s, 64 ms per text, 2.2 GB peak

What I'd add for real production: queue timeout with `Retry-After` · auth + rate limiting · Prometheus metrics · a larger quality eval · or Hugging Face TEI instead of a hand-written server

Demo

1. Swagger at `/docs`
2. A validation error, and its request id in the logs
3. `examples/limits.py` : truncation flag and token budget
4. `examples/demo.py` : Danish, English and German side by side